

Polar Cloud Detection: A Pixel-Level Classification Pipeline for MISR Satellite Imagery

1 Introduction

Cloud detection in polar regions is challenging because both clouds and ice/snow-covered surfaces appear white and cold in satellite imagery, making traditional detection methods ineffective. This study uses data from NASA's Multi-angle Imaging SpectroRadiometer (MISR) sensor, which captures images at nine different viewing angles.

My dataset consists of 164 satellite images, with only three containing expert labels that identify pixels as either cloud or non-cloud. The report is divided in three parts. First, I perform exploratory data analysis to understand the structural patterns distinguishing clouds from non-clouds; second, I engineer effective features for cloud detection and implement transfer learning using autoencoders pre-trained on unlabeled images; finally, I develop and evaluate several classification models, selecting the most effective one for cloud detection.

My final goal is to build a prediction model that can accurately distinguish cloud from non-cloud pixels in polar satellite imagery, contributing to better algorithms.

2 Exploratory Data Analysis

In this chapter, I will perform exploratory data analysis on the MISR satellite images provided, visualizing cloud patterns using expert labels, investigating relationships between radiance measurements across different angles, splitting my data strategically to support robust model development, and addressing data quality issues to ensure reliable inputs for my cloud detection algorithms.

2.1 Visualization of Expert Labels

I begin my analysis by visualizing the expert labels for the three labeled images (O013257, O013490, and O012791) to gain an understanding of the spatial distribution of clouds and non-clouds in the dataset. Each image contains approximately 115,000 pixels (O013257 with 115,000 pixels, O013490 with 115,032 pixels, and O012791 with 114,973 pixels). Figure 1 shows the expert labels mapped according to their X and Y coordinates.

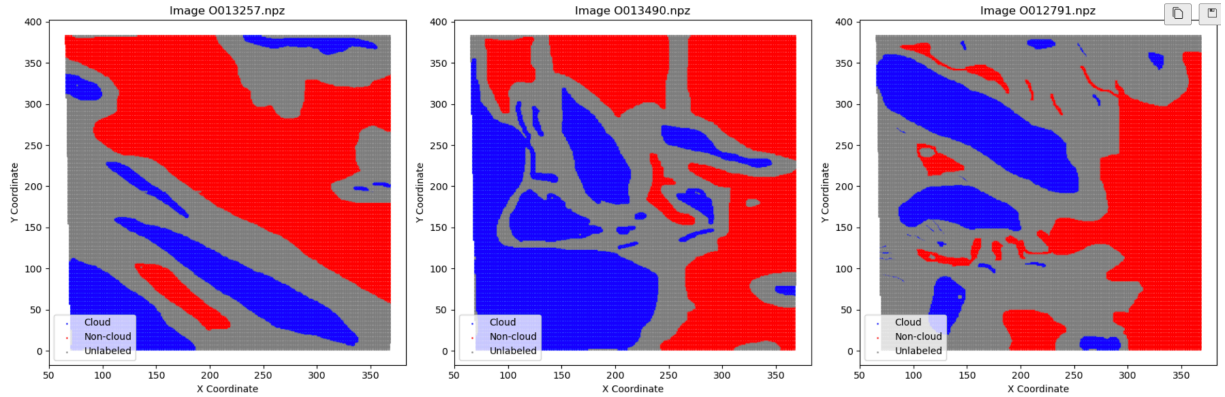


Figure 1: Expert Labels for the Three Labeled Images

The above spatial visualization reveals distinct patterns in cloud formation across the three images: Image O013257 shows significant non-cloud regions (red) in the upper portions, with cloud formations (blue) primarily in the lower sections of the image. There are also substantial unlabeled regions (grey) throughout the image; Image O013490 shows a more balanced distribution, with larger cloud formations in the lower half and non-cloud areas predominantly in the upper half; and Image O012791 shows significant cloud coverage (blue) in the center of the image forming band-like patterns, with non-cloud areas (red) along the edges.

These patterns suggest that clouds in polar regions form connected structures rather than appearing as random, scattered pixels across an image. The significant presence of unlabeled regions in all three images highlights the challenge of getting thorough expert labels in remote sensing applications.

2.2.1 Relationships Between Radiances at Different Angles

Next, I examine the relationships between radiances at different angles to understand how they might help differentiate clouds from non-clouds. My dataset contains measurements from five different angle perspectives: DF (70.5° forward), CF (60.0° forward), BF (45.6° forward), AF (26.1° forward), and AN (nadir, 0°). From the three labeled images, I identified a total of 207,681 labeled pixels (70,826 from O013257, 82,083 from O013490, and 54,772 from O012791).

The scatter plots in Figure 2 indicates that cloud pixels (blue) generally have higher radiance values across all angles compared to non-cloud pixels (red), with cloud pixels showing greater spread and variability. Both classes displays strong linear relationships between measurements taken at different angles, but form distinguishable clusters. The diagonal plots (density distributions) show that cloud pixels have wider and more variable distributions compared to the tighter distributions of non-cloud pixels.

Pairwise Relationships: Cloud vs. Non-Cloud Radiances

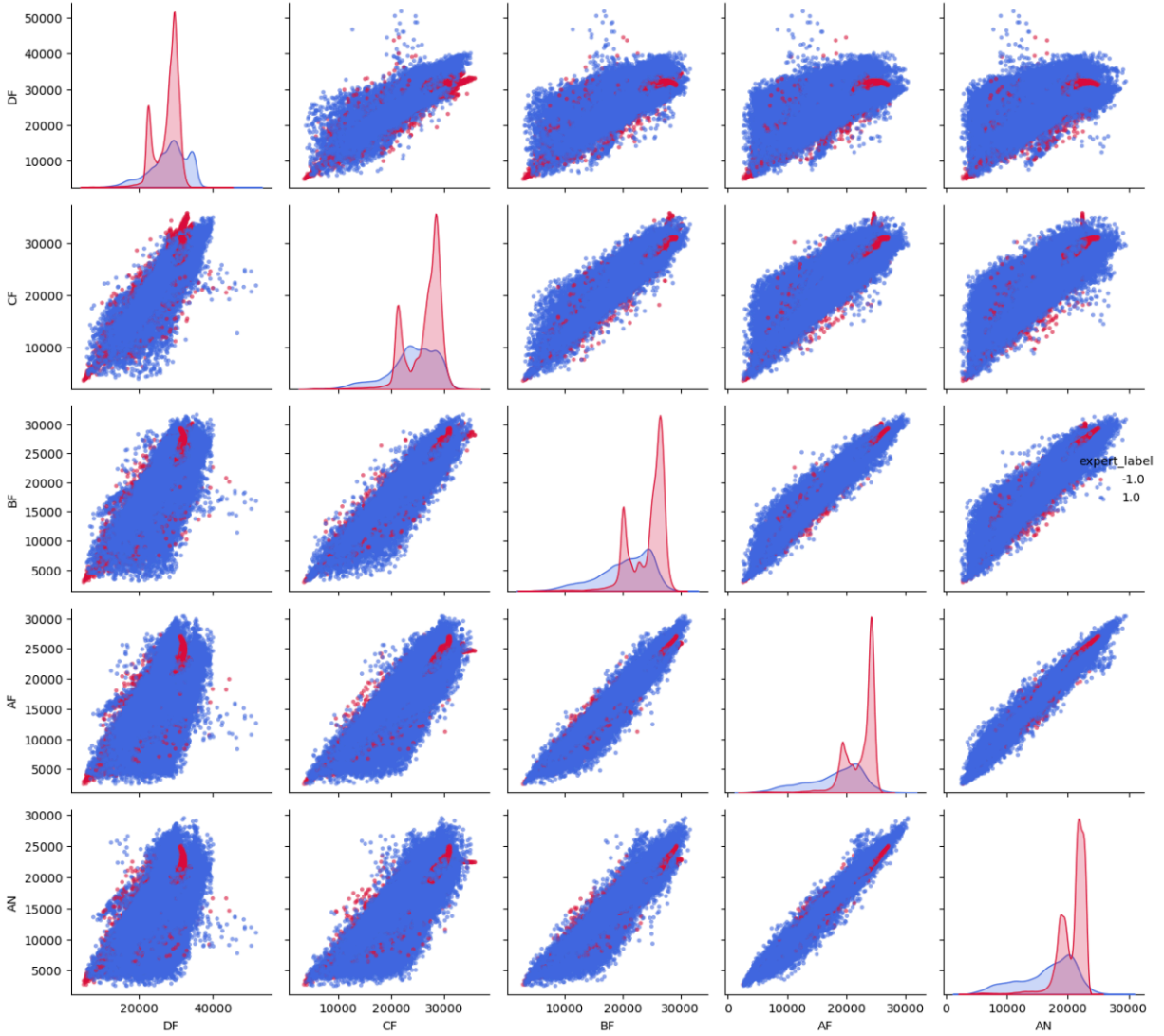


Figure 2: Pairwise Relationships Between Radiance Measurements (Cloud vs Non-Cloud)

To quantify these relationships, I computed correlation matrices for cloud and non-cloud pixels separately. The correlation matrices in Figure 3 shows that for cloud pixels, the correlations between angles range from 0.52 to 0.99, with adjacent angles (like AF-AN with 0.99) showing the highest correlations while for non-cloud pixels in figure 4, correlations are generally higher across all angle pairs, ranging from 0.82 to 0.99. The most significant difference is in the correlation between DF and AN, which is 0.52 for cloud pixels but 0.82 for non-cloud pixels.

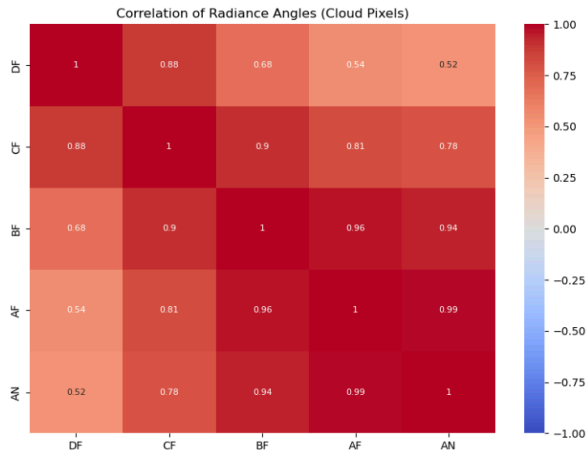


Figure 3: Correlation Matrix for Cloud Pixels

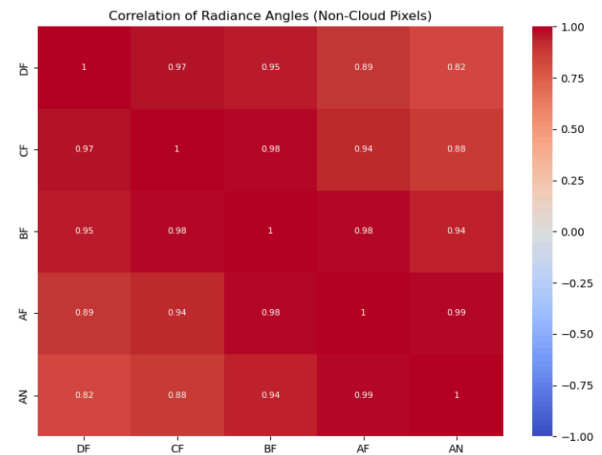


Figure 4: Correlation Matrix for Non-Cloud Pixels

This pattern directly supports the understanding described in Yu et al. (2008) that ice and snow surfaces scatter radiation more isotropically than clouds. When a surface scatters radiation isotropically (more evenly in all directions), the measurements from different viewing angles will be more consistent, resulting in higher correlations. In contrast, clouds exhibit more complex and anisotropic scattering behaviors, particularly when comparing extreme angles like DF (70.5° forward) and AN (nadir, 0°).

2.2.2 Analysis of Feature Characteristics

I further analyze the three features highlighted in the Yu et al. (2008) paper: NDAI (Normalized Difference Angular Index), SD (Standard Deviation), and CORR (Correlation). These features were specifically developed to help in polar cloud detection.

The statistical analysis reveals clear differences between cloud and non-cloud pixels:

Feature	Class	Mean	Std Dev	Min	Max
NDAI	Cloud	0.264589	0.126909	-0.348311	0.816864
	Non-cloud	0.142714	0.043273	-0.172772	0.693426
SD	Cloud	723.74166	529.91462	44.707161	6516.2803
	Non-cloud	163.76302	443.07198	13.329613	7251.0093
CORR	Cloud	0.413331	0.383378	-0.888728	0.980773
	Non-cloud	0.366549	0.422800	-0.942551	0.982818

Key observations:

- NDAI values for cloud pixels are significantly higher (mean: 0.26) than for non-cloud pixels (mean: 0.14), with cloud pixels also showing greater variability.
- SD is substantially higher for cloud pixels (mean: 723.74) compared to non-cloud pixels (mean: 163.76), indicating greater textural variability in cloud surfaces.
- CORR shows a smaller difference between the classes, with cloud pixels having a slightly higher mean (0.41 vs. 0.37).

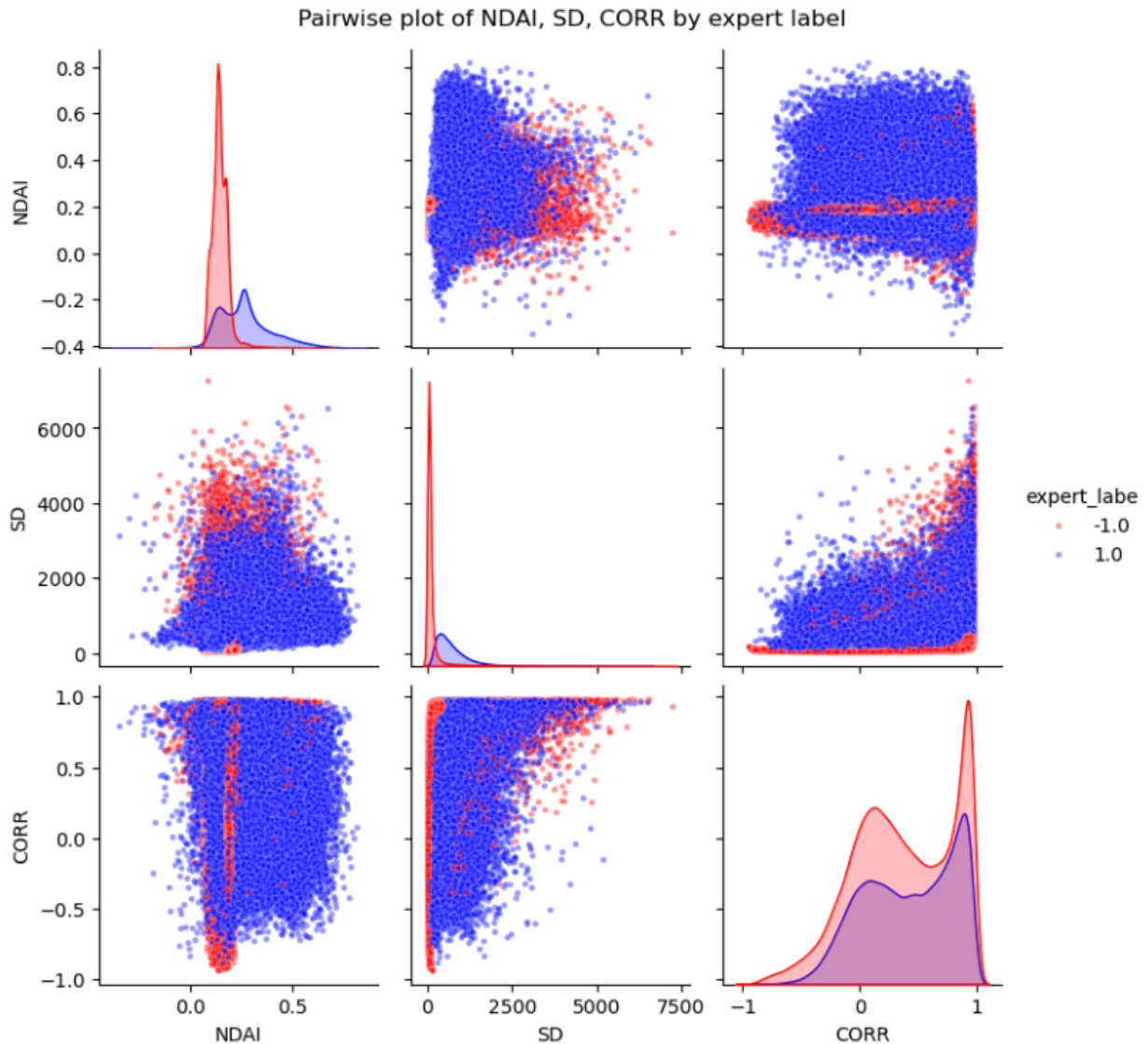


Figure 5: Pairwise Relationships Between NDAI, SD, and CORR by Expert Label

The pairwise plots in Figure 5 reveal that the combination of features, particularly NDAI and SD, provides enhanced classification power. In NDAI vs. SD plot, I observe that non-cloud surfaces cluster predominantly in the region of low NDAI and low SD values, while cloud pixels spread toward higher values in both dimensions. This creates distinct regions in the

feature space that can be leveraged for classification. These differences align with Yu et al's approach of identifying surface characteristics rather than cloud characteristics which proves particularly valuable in polar regions where traditional cloud detection methods struggle due to their similar brightness and temperature characteristics of clouds and snow/ice surfaces.

2.3 Data Splitting Strategy

I implement a data splitting approach to ensure stable model evaluation. First, I perform an image-level split by designating two of the three labeled images (O012791.npz and O013257.npz) for training purposes and take the third image (O013490.npz) as my independent test set. I do not use a random-split on the data, because later in the report, I use some features which reference information on the pixels in the surrounding. If I were to use a random-split, I would have great dependencies between the train and test data, which I do not have when I split by the images.

Within the training images, I further subdivided each image into four quadrants (Q1-Q4) based on the median x and y coordinates specific to each image. I defined these quadrants as:

- Q1: Pixels where $x \leq \text{median_x}$ and $y \leq \text{median_y}$
- Q2: Pixels where $x > \text{median_x}$ and $y \leq \text{median_y}$
- Q3: Pixels where $x \leq \text{median_x}$ and $y > \text{median_y}$
- Q4: Pixels where $x > \text{median_x}$ and $y > \text{median_y}$

This quadrant structure supports potential spatial analysis and provides a framework for evaluating model performance across different regions of the images for the cross-validation of the models. Prior to model training and testing, I filter out all unlabeled pixels, retaining only those with definitive expert labels (cloud or non-cloud). I also removed the spatial coordinate columns (x, y) and image identifiers from the feature set after creating the quadrant assignments, as these would not provide a general information for the cloud detection task.

This approach gives a clear separation between training and testing data, preventing data leakage along with assessment of model generalization to entirely unseen geographical regions.

2.4 Data Cleaning and Preprocessing

When dealing with real world data, these images may have imperfections that can affect analysis quality. My cleaning approach focuses on:

- **Outlier Detection Analysis:** I examined the distribution of my key features to identify potential outliers, applying a z-score threshold of 3 separately for cloud and non-cloud pixels as they have different distributions. This analysis identified approximately 6,236 outliers (about 3% of the dataset), with a slightly higher percentage in non-cloud pixels (3.33%) compared to cloud pixels (2.49%). The SD feature showed the most significant impact from outlier removal, with maximum values dropping from 7251 to 2313, and non-cloud means decreasing by about 40%. The NDAI feature range also contained some extreme values from -0.348 to 0.817, though with less impact on the overall distribution. The CORR feature showed relatively stable distributions with fewer extreme values.
- **Missing Value Handling:** I examined the dataset for missing or invalid values that could impact my analysis. No missing values (NaN/null) were found in any of the columns. Additionally, I checked for implausible measurements, specifically looking for negative or zero radiance values that would indicate sensor errors, but none were detected. This suggests the expert-labeled dataset was already pre-processed to include only valid measurements.

After this exploratory analysis of data quality, I determined that the identified outliers represented a small portion of the overall dataset. Rather than removing these observations, I chose to retain the complete dataset for my subsequent feature engineering and modeling steps. This decision was based on the understanding that outlier values in features like SD might still contain meaningful signal for distinguishing cloud patterns, especially since SD emerged as an important feature in Yu et al. (2008) and in my previous analyses.

3 Feature Engineering

In this chapter, I explore feature engineering techniques to enhance my cloud detection model, analyzing the relative importance of existing features, creating new features, and implementing transfer learning with autoencoders to extract deeper patterns from the MISR satellite imagery.

3.1 Feature Importance Analysis

Before I deal with the modeling and classification of individual pixels, I want to take a closer look at the current features of my data set and, in particular, at the importance of existing features. Such an importance of existing features can be carried out using a CART model, for example. The CART model creates a decision tree and has the special feature that only two branches may be present. This makes the model relatively easy to interpret, which is why it can be used here for the initial determination of feature importance. It was determined that a

total of only 4 levels are permitted in order to facilitate the complexity and thus the interpretability of the CART model.

Figure 6 shows the decision tree on the original data set. Here it is clear that the attributes SD and CORR, which are the expert features, have the highest feature importance, followed by the other features.

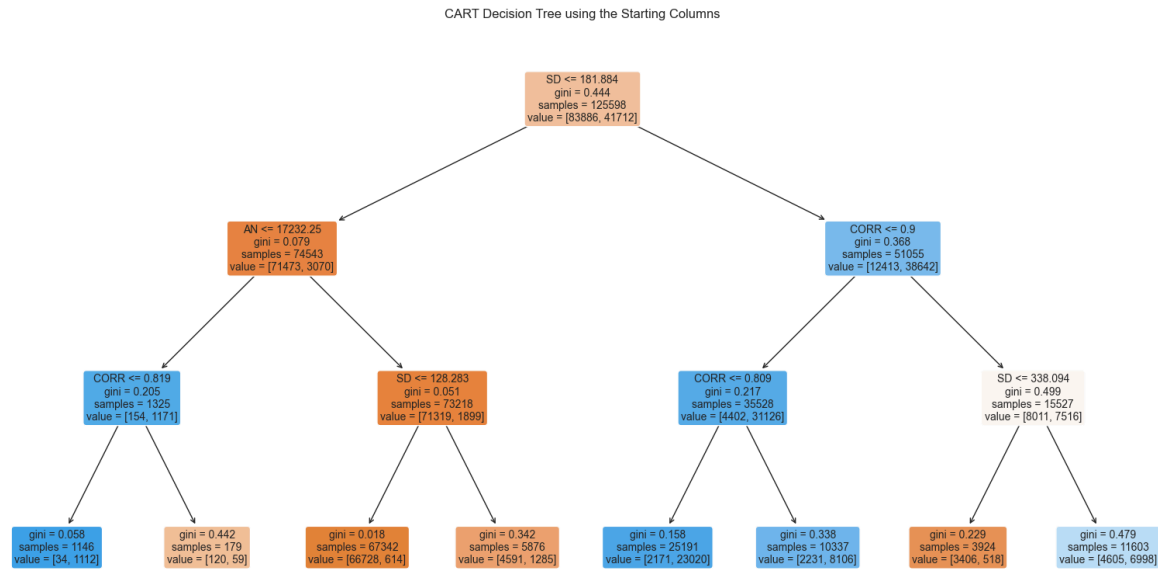


Figure 6: CART Model before Feature Engineering

I can also see that an accuracy of 95% can already be achieved with this simple CART model, which is already an outstanding performance for such a “simple” model. Nonetheless, in the next chapters, I will continue to engineer more features and at the end I will set up a corresponding CART model to show the differences between the original dataset and the newly created dataset with the new features.

3.2 Feature Creation

Now, I will take a closer look at which new features I can add to the data set for better prediction. The basic idea here is based on the findings of the EDA, where I can clearly see on the images that clouds form in larger clusters and these clusters then always form a wave/cloud front together. This means that the pixels in my surroundings are usually also clouds, which means that if I integrate this environmental information into my current pixel/data point, I obtain more in-depth information about the surroundings and can take this into account in the classification. The only disadvantage that arises is the choice of train and test split, since a single data point now already contains information about the

environment and therefore no random split may be made, but entire regions/images must be assigned to either the train or the test data set.

Now to the exact implementation of the above principle. For each of the existing features such as SD, CORR, NDAI, DF, CF, BF, AF and AN, a variant is now calculated for the mean value, max value and min value from the environment. After several attempts, the environment was calculated with a 3x3 window, a 5x5 window and a 9x9 window, whereby the mean, min and max values were calculated from the 80 surrounding points in the last area.

After this step, the data set has therefore grown significantly, as one NDAI feature has now become a total of 9 additional NDAI features with NDAI_3_Mean, NDAI_3_Min, NDAI_3_Max, NDAI_5_Mean, NDAI_5_Min, ..., NDAI_9_Max. This data set will therefore grow to over 80 features as a result of this step, which means that a large number of features are available across the environment, which must be reduced again in the modeling if necessary.

3.3 Transfer Learning with Autoencoders

I attempt to develop new features that efficiently aggregate information including surrounding pixels using autoencoders. There are a wide range of hyperparameters, including the structure of the autoencoder (such as the number of layers), learning rate, and number of embeddings, and tuning all of these using grid search is computationally too expensive, so I first determined the structure of the autoencoder and learning rate.

I set up three patterns for the structure of the autoencoder: a model with three fully-connected layers ("Fully Layer model"), a model with two fully-connected layers and one 2D convolutional layer ("Combined model"), and a model with one fully-connected layer and two convolutional layers ("CNN model"). I also set up three patterns for the learning rate: 0.001, 0.005, and 0.01, and calculated the validation loss (mean squared error) for the nine combinations. As a result, I found that Fully Layer model with 0.01 learning rate was the best.

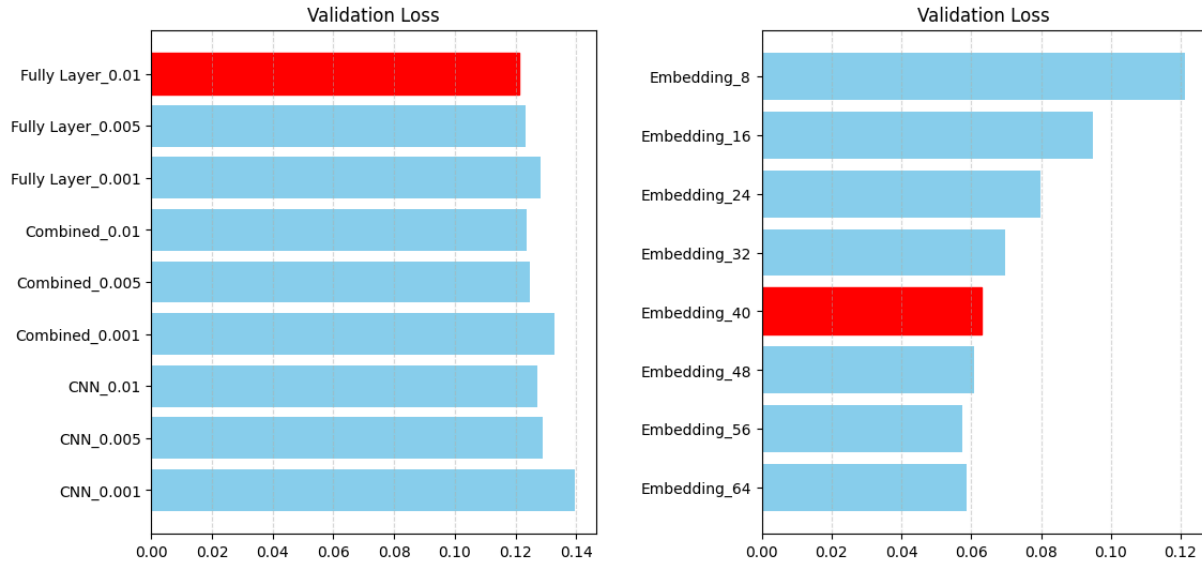


Figure 7: Comparison of Validation Loss

In addition, I tried nine different numbers of embeddings from 8 to 72, and found that increasing the number of embeddings to 40 greatly reduced the validation loss, but the subsequent decrease was limited. Considering the negative effects on modeling of increasing the number of less informative features, I decided on 40 as the number of embeddings. As a result of these efforts, I was able to reduce the validation loss to 0.06, almost the half of the initial score.

To avoid the data leakage, the images used for training and those used for validation were completely separated (the same image was never used for both training and validation, which is different from the provided code. As autoencoders use information from surrounding pixels, if this method is not used, data leakage will occur to a certain extent). In addition, given that the modeling part uses some unlabeled images for prediction, I trained the autoencoder using 132 of the 161 unlabeled images, and obtained the embeddings of the labeled images. The reason this process is called transfer learning is that, after training one model (autoencoder) with unlabeled images, the labeled images are fed into the model to obtain embeddings and improve the performance of another model (classifier).

From now on, I will refer to the features obtained through this process as "ae1" to "ae40".

3.4 Summary

To summarise the steps taken in feature engineering; in the first chapter, I saw that the original features with the CART model already enable relatively good performance and that the two features SD and CORR are particularly important. In chapters 3.2 and 3.3, I have now created further features using the surrounding pixels and an autoencoder. To conclude this

chapter, I will now analyze the feature importance with the CART model again and observe the changes there.

The next figure shows the result, which is performed on the old and on the new features. It can be seen that the differentiation of the clusters is hardly determined by the “old” features, but is mainly based on the new features from 3.2 and 3.3. It can also be seen that the features from the surrounding pixels have the greatest importance and are located relatively high up in the tree. The features of the autoencoder are present in the tree, but at a lower position.

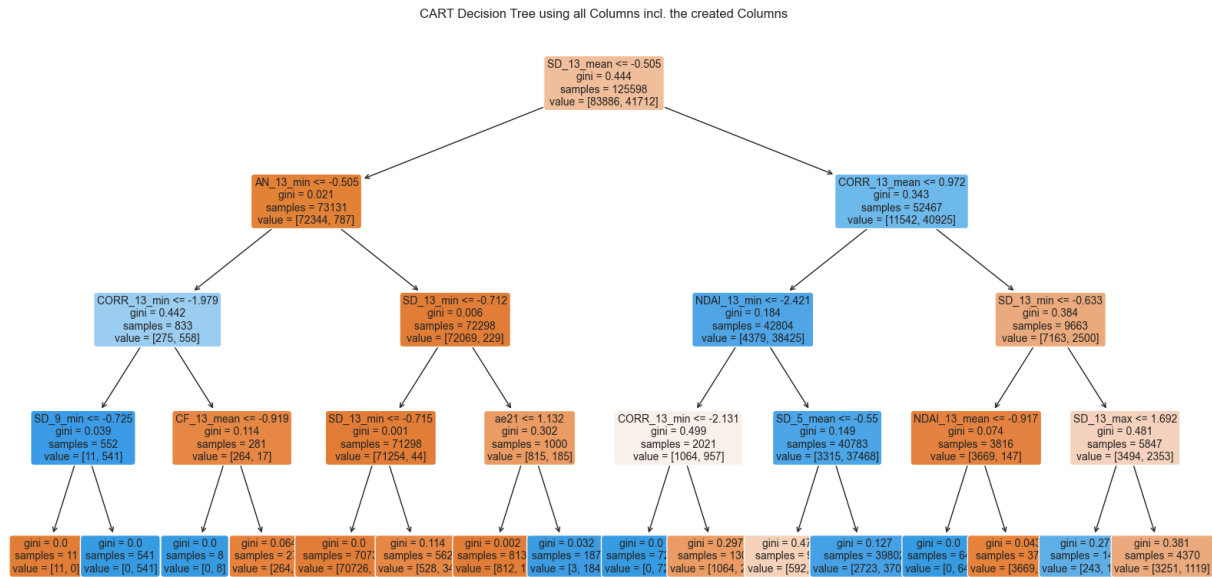


Figure 8: CART Model after Feature Engineering

Really interesting and a little surprising is, that the CART model gets an accuracy on the test dataset of 98% only based on that tree. Such a high accuracy shows me that the detection of clouds is possible with the combination only of that few data points and guarantees still an incredible performance on the test set.

Overall, it can be said after this analysis that the feature engineering from this chapter has once again greatly expanded and modified my data set. Fortunately, the analysis also shows that the steps from 3.2 and 3.3 have led to the addition of features with great importance, which should make classification in the next step much easier and produce significantly better results.

4 Predictive Modeling

In this chapter, I will take a closer look at the modeling and examine the corresponding model results. For all models, I assume that the prediction of non-cloud and cloud is equally important, which allows me to use the standard accuracy and loss functions in the models.

In addition, the training and testing split is performed as explained in the previous section 2.3 (the two images ("O13257.pnz", "O12791.pnz") are divided into four parts and cross-validation is performed, with the other image ("O13490.npz") used as test data). For k-cross-validation, I have introduced a column called Quadrant, which can be used to perform a corresponding 4k cross-validation by always using one quadrant as validation and three quadrants as training. This also means that the test and validation data sets are only slightly dependent on each other due to the edge pixels of the quadrants, but are otherwise completely independent of each other and can therefore still achieve good results.

In the following, I decided to program a Random Forest, a K-Nearest Neighbor (KNN) classifier, and a LightGBM. I decided not to use a complex model such as a deep neural network (DNN), as the performance of this model was significantly worse than the above models in initial tests. In the subchapters, I discuss the functionality of each model, the strengths of the model, the reasons for the selection, and the performance on my dataset.

4.1 Model Development and Assessment

4.1.1 Random Forest

In this case, I choose a random forest algorithm as one of my classifiers because I already see such good results with the CART model in Feature Engineering, which signaled to me that a relatively easy differentiation is possible with a tree-based method.

A random forest is an ensemble learning method that builds multiple decision trees during training and merges their predictions to produce a more robust and accurate final result. Each tree is trained on a randomly sampled subset of the data (with replacement) and considers only a random subset of features when determining splits. This approach introduces diversity among the trees, reducing the likelihood of overfitting and increasing overall generalization. In classification tasks, the final prediction is made by aggregating the outputs of the individual trees. Random forests are particularly effective for complex datasets because they capture non-linear relationships and interactions between features while maintaining interpretability through analysis of feature importance.

To identify the best set of variables for the random forest, I perform a hyperparameter search using the 4k cross-validation described above, where I try different values for the parameters

"n_estimators", "max_depth", "min_samples_split", "min_samples_leaf", and "bootstrap". In the end, I see that I still get the best results when I use a max_depth = 4 and n_estimators = 20, which means that the decision tree also performs best when I use only a really simple version of it.

With this parameterization, I get the following confusion matrix:

Accuracy: 98.15%

	Actual: Unclouded	Actual: Clouded
Prediction: Unclouded	41,516	1,314
Prediction: Clouded	207	39,046

This shows that I have an accuracy of 98% on the test dataset, which is quite impressive and shows a good performance at all on the test dataset, which shows me that the model is neither overfitted nor underfitted. The interesting part is that this performance of the best decision tree is still slightly worse than the CART algorithm on the full dataset.

4.1.2 K Nearest Neighbors

k-Nearest Neighbors (kNN) is a non-parametric supervised machine learning algorithm that is commonly used for classification and regression tasks. This model is attractive because, due to its heuristic nature, it uses very few assumptions. This makes my results much more interpretable and trustworthy. However, the simplicity is at the cost of model performance which is shown in my comparison between models.

The process begins with model selection. First, I choose the number of neighbors to consider (k). Then, I evaluate the Euclidean distance between every data point in my validation set with every point in my training set. For the first k datapoints in my training set closest to my validation datapoint, I identify the most common class among the nearest k labelled datapoints from the training set. This class is then my predicted value for the unseen validation datapoint in my validation set. After all validation points are assigned a predicted class, I evaluate the proportion of correctly labelled datapoints in my validation set. I then evaluate the prediction accuracy across a range of k values to obtain the optimal k value and my finalized knn model. This knn model is then implemented on my test set to get my final prediction accuracy.

To explore hyperparameter tuning in more detail, I used 4-fold cross validation. I utilized the GroupCV method of the sklearn.ModelSelection to find the optimal k value across each fold independently. The idea is that I find the k value that maximizes testing accuracy across each fold to maintain generalizability of my model. This ensures that my model does not overfit to

a given fold and has generalizability to unseen data. The prediction accuracies for each fold are shown below:

Fold	Test Score
Split 0	0.8925
Split 1	0.7164
Split 2	0.9994
Split 3	0.8228

I arrive at $k = 2$ as the optimal k value with an average testing accuracy of 76.47% across the four folds of my training/validation set in 4-fold CV. With this parameterization using my highest performing model with $k = 2$, I achieve a 94.45% prediction accuracy. This is shown in the following confusion matrix:

Accuracy: 94.45%

	Actual: Unclassified	Actual: Classified
Prediction: Unclassified	40,912	1,918
Prediction: Classified	2,640	36,613

4.1.3 Light Gradient Boosting Machine ("LightGBM")

LightGBM is a model that aims to improve the performance of decision trees by using gradient boosting. One of its features is that it uses both training and validation data during the training process, and it repeatedly learns to reduce the error between the predicted and actual values of the validation data. The disadvantage of this approach is that the amount of calculation increases in proportion to the amount of data, but LightGBM speeds up this process using various methods. As with random forest, if the data is unbalanced (the number of cloudy cells and non-cloudy cells differs greatly), I need to do specific adjustments to improve accuracy, but in this analysis, no such problems were observed.

For hyperparameter tuning, I conducted cross-validation using four quadrants with the number of leaves as a candidate for [15, 31, 63] and the learning rate as a candidate for [0.05, 0.1, 0.15], and the number of leaves was selected as 31 and the learning rate as 0.15. In addition, for training the final model, I trained the model using the data in the fourth quadrant as the validation data.

At this point, I also noticed that the LightGBM model generally performs worse in terms of accuracy in the test/train split and cross-validation with more features. After several tests, I then decided to drop all features of the variables "AF", "BF", "CF", "DF" and "AN", resulting in

an accuracy increase from 98.18% to 98.74%. This feature selection step only led to a significant improvement in the LightGBM model, which is why it was only implemented there.

The results of the fitting to the test data are as follows. The accuracy of 98.74% is the highest of the three models (in the next section, I will discuss the comparison of the models, including this point).

With this parameterization, I get the following confusion matrix:

Accuracy: 98.74%

	Actual: Unclassified	Actual: Classified
Prediction: Unclassified	41,579	1,031
Prediction: Classified	1	39,009

4.2 Best model, Diagnostics and Feature Importance

When comparing the performance of the three models on the test data, all of them performed extremely well, but LightGBM slightly outperformed the other two models.

	Random Forest	KNN	LightGBM
Accuracy	98.15%	94.45%	98.74%

However, choosing the best model based solely on the prediction performance for a single image can lead to neglecting the risk of overfitting. Therefore, I also check the results of the 4K cross-validation using the training data (when the four quadrants are highly independent, the results of cross-validation suggest the degree of generalization performance of the model for other images). As a result, I can also see that LightGBM consistently demonstrates higher performance than random forest and KNN. This suggests that LightGBM predictions are effective for identifying clouds using satellite images.

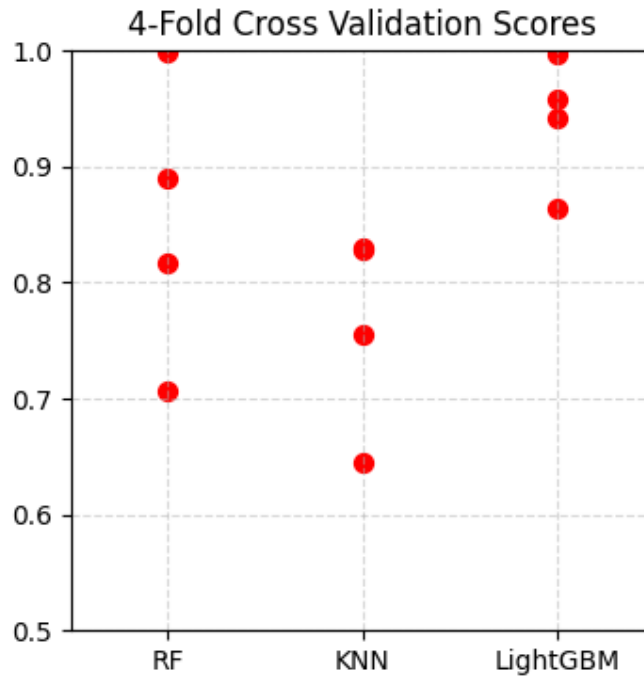


Figure 9: Comparison of the Cross Validation Scores

Next, let's check the convergence of the model. The plot of the training curve shows that the error rate decreases as training progresses, and that the model has finally converged sufficiently.

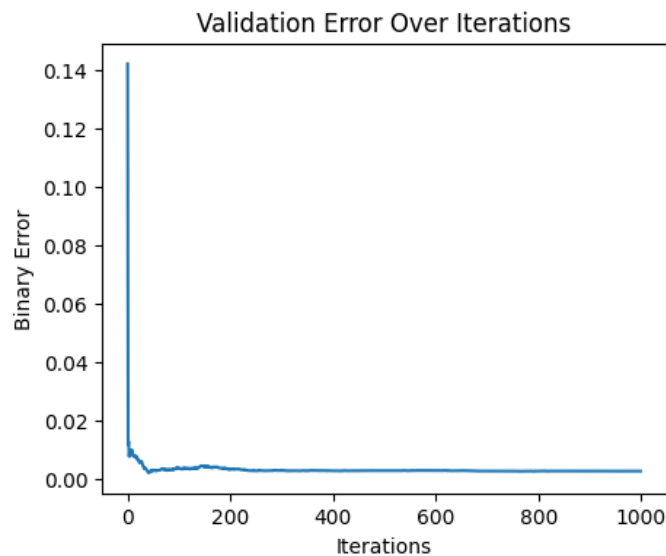


Figure 10: Convergence of the model

Next, let's check the feature importance and permutation importance to see what variables play a major role in the LightGBM prediction model. For both evaluation criteria, I can see

that the variable “SD_13_mean” has a high degree of importance. This is consistent with the previous discussions that the distribution of SD varies largely for each cloud label. In addition, the fact that “SD_13_mean” was selected rather than “SD” strongly suggests that feature engineering using surrounding variables has led to improved the performance of the model.

If you check the other variables, you can see that the features engineered by the autoencoder starting with “ae” also contribute to a certain extent. In addition, it should be noted that none of the features I was given in advance had a high degree of importance. This suggests that the features I engineered clearly outperform existing features by making effective use of information from surrounding pixels.

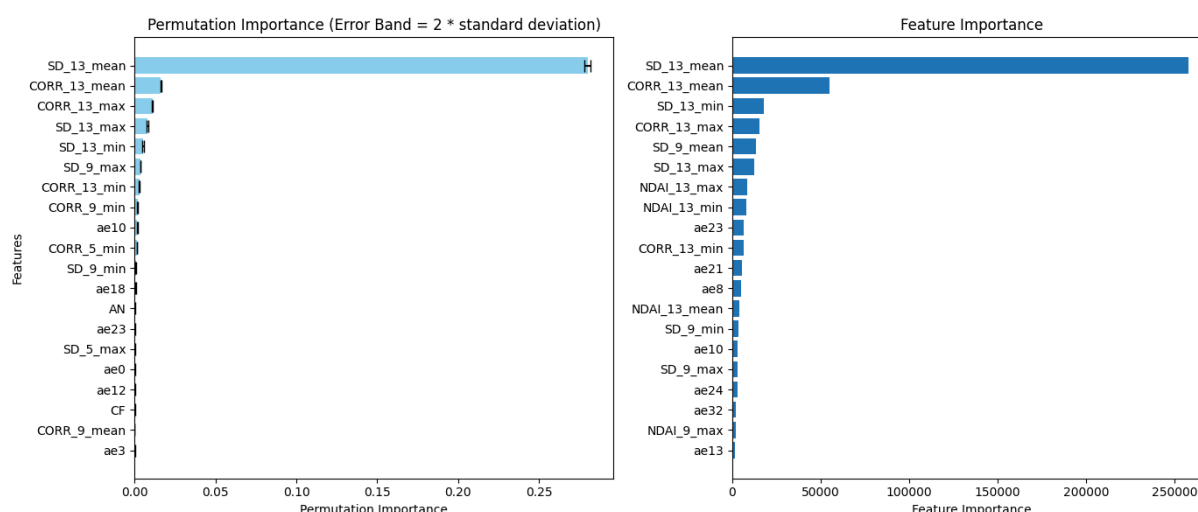


Figure 11: Feature importances of the lightGBM

4.3 Post-hoc EDA and Error Analysis

As post-hoc EDA, let's compare the prediction results of the modeling using lightGBM with the actual expert labels. Here, I show the two results of lightGBM discussed in the previous section. As the high accuracy shows, they are quite similar to the expert label, but I can see that the predictions and the expert label differ in (i) the edges and (ii) the center part of the images.

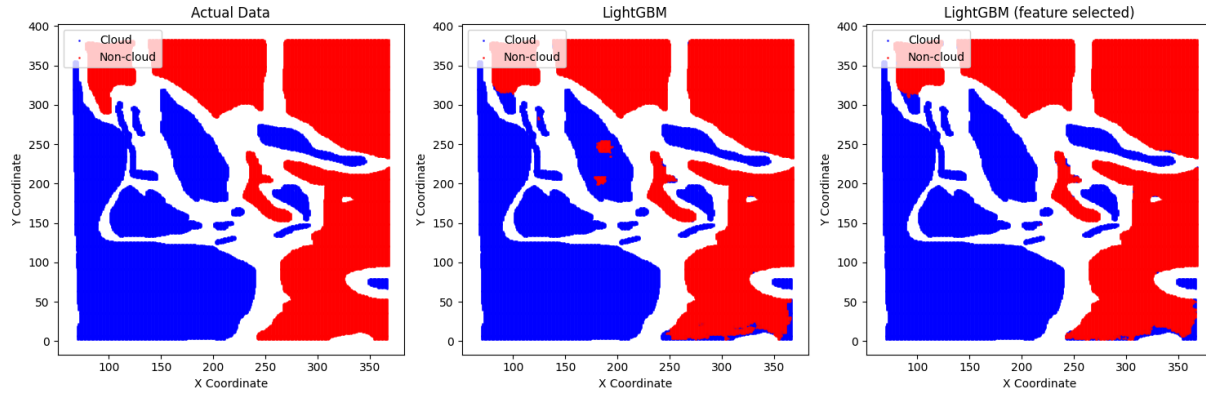


Figure 12: Comparison between the expert label and the prediction

The prediction errors at the edges of the image can be explained by the information constraint that it is not possible to obtain all the information from the surrounding pixels. In fact, when patches are created using autoencoders, etc., if there are missing values in part of the patch at the edge of the image, the image is flipped and the missing values are filled in, and the impact of errors caused by this method cannot be ignored.

The prediction error in the center of the image can be discussed in relation to the main variable "SD_mean_13". Looking at the distribution of this feature value for the test data, the center part is colored red, which is similar to the area without clouds. For the remaining areas, "SD_mean_13" is similar to the actual expert labels, so it can be said that this feature has high performance as a variable, but it does not have the performance to predict all the expert labels. Interestingly, I can see that the prediction results for this central part improve when the feature values are reduced. This suggests that the reduction of feature values improves the balance of variables contributing to the model.

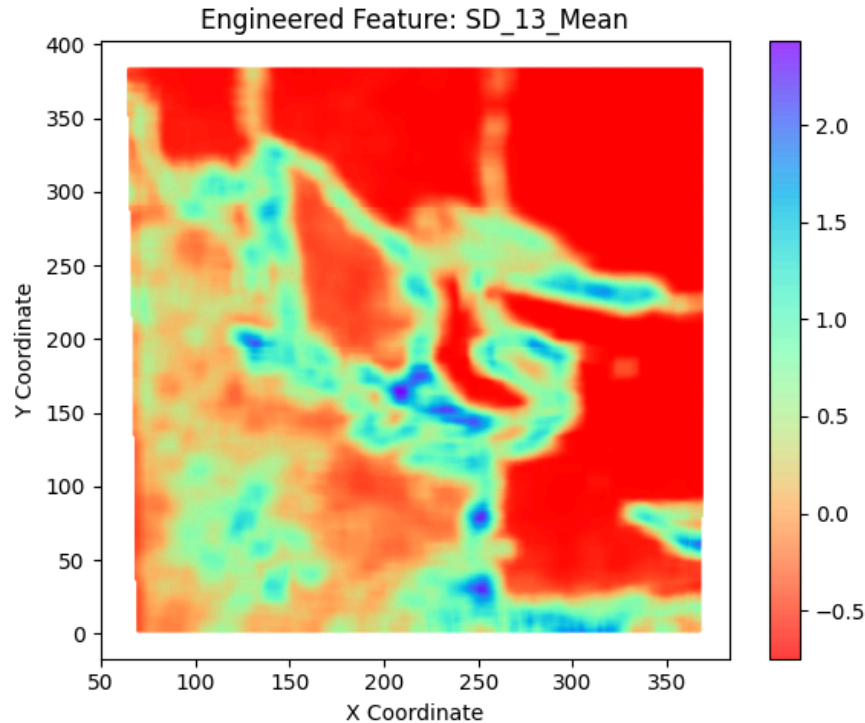


Figure 13: the Distribution of the Engineered Feature "SD_mean_13"

4.4 Model Generalization and Stability

In order to evaluate the effectiveness of a predictive model for future data, I can approach it from the perspective of how stable the model's conclusions are in the face of changes in the data, and whether the results of the survey are independent on the assumptions I used in the analysis.

As a first approach, I added noise to the data in two ways.

First, let me add noise to the variables used in the prediction (e.g., engineered features). I add random noise generated from a normal distribution to 10% of each data point (the variance of the random noise is equal to the sample variance of the feature values). As a result, I can see that there is almost no effect on the accuracy, although there is a slight increase in misjudgments in the middle part of the data.

Next, I consider adding noise in the same format to the observation data and feature values I had at the time I started the analysis. After adding noise to the data, I engineered features in the same process, including autoencoder features (as the autoencoder has been trained using unlabeled data, there is no need to relearn the model), and then made predictions using LightGBM. Looking at the results, I can see that, unlike the previous results, the predictions have not maintained their accuracy at all. However, this does not only point to the low

stability of this particular prediction model. When random noise is added to the data at a rate of 10%, errors propagate during the process of generating features while incorporating information from surrounding data. Therefore, this result suggests that the approach of utilizing the surrounding pixels has a structural weakness in terms of stability in that the effects of data observation errors and other factors propagate over a wide range.

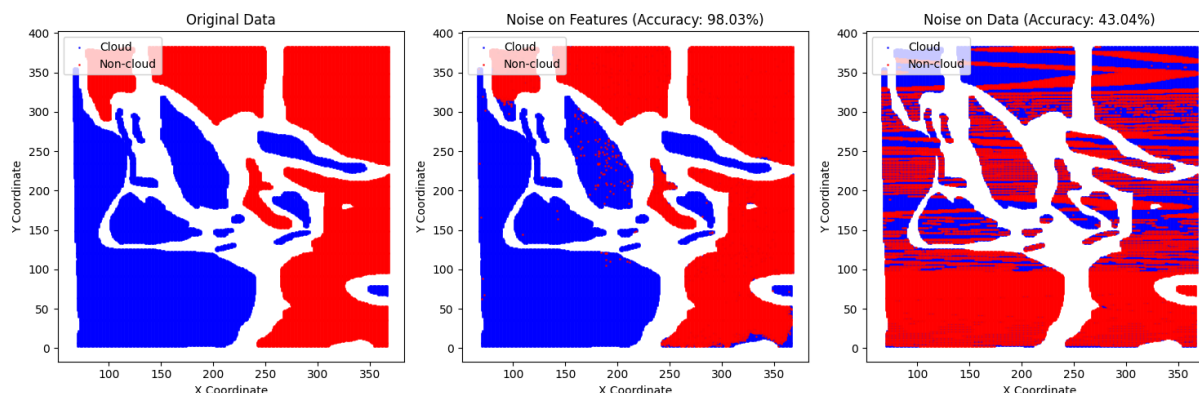


Figure 14: Stability Check: Add Noise on Features and Data

Finally, I will examine the major assumption I used in my predictions that I used “O13490.npz” as the test data, out of the three images with expert labels. After tuning the parameters and making predictions in exactly the same way as before, I can see that the accuracy obtained from the two images (“O13257.npz”, “O12791.npz”) is slightly lower than the original result. However, the fact that the correct answer rate is over 85% in all cases where the experiment was conducted using either image as test data can be said to support the fact that there is a certain degree of stability in the prediction model using LightGBM.

Image_ID (test data)	O013490.npz (default)	O013257.npz	O012791.npz
Accuracy	98.74%	90.66%	85.90%

4.5 Sanity Check

Finally, as a sanity check, let's check the results of the predictions I made for the image without a label, after obtaining the feature values. Note that the image used in this case (“O120204.npz”) was not used for training the autoencoder.

Since it is difficult to discuss the validity of the model based on the results of LightGBM alone, as there are no expert labels on this image, let me evaluate the validity by showing the results of all three prediction models. The results of the three prediction models are similar in terms of the general direction, and they all conclude that there are no clouds in the upper left and

there are clouds in the lower part. However, it is also important to note that the three models' predictions differ slightly in their conclusions for the central part of the image (LightGBM can be said to be a result that is somewhere between Random Forest and KNN, and it can also be interpreted as a moderate conclusion).

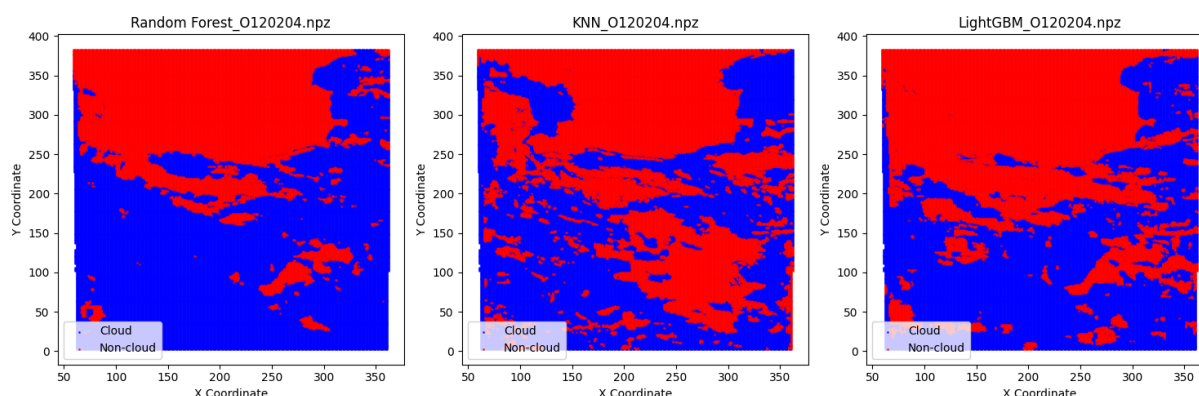


Figure 15: Sanity Check: Prediction Results for unlabeled data

As a conclusion of the sanity check, while it is positive that the three prediction models are likely to be able to capture the general trend, it also shows that the high performance obtained with the test data may not necessarily be obtained for all images. There are multiple reasons for this, but one of the major factors is that there are only three images with expert labels (and only two images for training data), so the risk of overfitting is potentially high (although it should be noted that I have improved the generalization performance using various methods, such as cross-validation with four quadrants).

5 Conclusion

In conclusion, I used the three models Random Forest, Nearest Neighbor and LightGBM for the classification of the clouds on individual pixels of the image. In order to enable a good classification, new features were created in a feature engineering step, which are based directly on the pixels in the environment on the one hand and on an autoencoder output, which also includes the pixels in the environment, on the other. These additional features resulted in dependencies between different surrounding pixels, which meant that a random split in the training process was no longer possible. It was therefore decided that the train/test split would be based on the three available images and the cross-validation split would be based on the quadrants. I can be sure that there is no dependency in the train/test split and only a slight dependency between the groups in the cross-validation split.

Overall, very good results were achieved using these methods, with the LightGBM model performing best on the train/test split and the cross-validation split. In addition, the LightGBM model also exhibits good stability with a permutation of the features. Nevertheless, the problem remains with all these methods that I have created dependencies on individual data points and thus encountered certain problems when evaluating the models. Overall, it can also be viewed critically that only 3 images with different cloud formations serve as the basis for the entire training and evaluation, which may mean that different cloud types with different variable values are only slightly covered, which can lead to problems with future images, although the model has very good performance on the existing labeled data.

6 Bibliography

Yu, Bin, Tao Shi, Eugene E. Clothiaux, and Amy J. Braverman. 2008. "Daytime Arctic Cloud Detection Based on Multi-Angle Satellite Data with Case Studies."